

Python Basics — Student Notes

2-Hour Session | Keep this handy while you practice

1. The Basics

- Python runs code line by line, top to bottom.
- `print()` displays output on the screen.
- Comments start with `#` and are ignored by Python — used to explain code.
- Variables store values using `=`

```
print("Hello, World!")
```

```
# This is a comment  
x = 5
```

2. Data Types & Variables

- **int** — whole numbers (e.g. 5, -3, 100)
- **float** — decimal numbers (e.g. 3.14, 2.0)
- **str** — text, written in quotes (e.g. "hello")
- **bool** — True or False

```
name = "Alice"  
age = 21  
is_student = True
```

```
print(type(name))    # <class 'str'>  
print(type(age))     # <class 'int'>
```

```
print(name.upper())   # ALICE  
print(name.lower())   # alice  
print(name.strip())   # removes extra spaces
```

Note: Use `type(variable)` anytime you want to check what kind of data it holds.

3. Control Flow: if / else

Used to make decisions in code based on conditions.

```
age = 18  
  
if age >= 18:  
    print("You can vote")  
else:  
    print("Not yet eligible")
```

- `==` means "equal to" (comparison, not assignment)
- Comparison operators: `==`, `!=`, `>`, `<`, `>=`, `<=`
- `and` / `or` can combine multiple conditions

4. Loops

for loop — repeats a set number of times, or over items in a list:

```
for i in range(5):
    print(i)
# Output: 0 1 2 3 4
```

while loop — repeats as long as a condition is True:

```
count = 0
while count < 3:
    print("Counting:", count)
    count += 1
```

- break — stops the loop early
- continue — skips to the next loop iteration

5. Functions

Functions let you reuse a block of code instead of repeating it.

```
def greet(name):
    return "Hello, " + name + "!"

print(greet("Alice"))
print(greet("Bob"))
```

- def starts a function definition
- Values inside () are called parameters
- return sends a value back to wherever the function was called
- You can set a default value: def greet(name="friend")

Note: print() just shows something on screen. return actually sends the value back so you can use it later.

6. Lists

An ordered collection of items, written in square brackets.

```
fruits = ["apple", "banana", "cherry"]

print(fruits[0])      # apple (indexing starts at 0)
fruits.append("mango") # adds an item

for fruit in fruits:
    print(fruit)
```

7. Dictionaries

Stores data as key : value pairs, written in curly brackets.

```
student = {"name": "Alice", "age": 21}

print(student["name"]) # Alice
```

```
for key, value in student.items():  
    print(key, ":", value)
```

8. Practice Task

Write a program that stores 3 friends' names and ages in a dictionary, then loops through and prints a sentence for each: '<name> is <age> years old.' Use a function to format the sentence.

9. Quick Reference Cheat Sheet

<code>print("text")</code>	<code># display output</code>
<code>x = 5</code>	<code># assign variable</code>
<code>type(x)</code>	<code># check data type</code>
<code>if / else</code>	<code># make decisions</code>
<code>for i in range(n):</code>	<code># loop n times</code>
<code>while condition:</code>	<code># loop while true</code>
<code>def name(params):</code>	<code># define a function</code>
<code>return value</code>	<code># send back a value</code>
<code>list = [a, b, c]</code>	<code># ordered collection</code>
<code>dict = {"key": value}</code>	<code># key-value pairs</code>

Homework: Build a simple contact book using a dictionary and functions.